

ClimbingGraph

Using Knowledge Graphs to Predict Climbing Grades and Recommend Routes



(AI Generated)



TECHNISCHE
UNIVERSITÄT
WIEN

Course: Knowledge Graphs

Student: Juan Moreno Segura

Mode: 6 ECTS

Date: June 12th, 2026

Project Overview and Motivation

The sport of rock climbing relies heavily on grading systems to communicate the difficulty of a route. However, climbing grades are subjective, what one person considers easy, another might find hard, a phenomenon commonly known as "sandbagging". Furthermore, when climbers are at a specific crag, finding routes of a similar style and difficulty that are nearby typically requires manually scrolling through hundreds of comments and routes in traditional databases and websites.

For years, climbers found new routes through printed guidebooks. Today, online databases offer route information, but they usually miss the big picture of how routes relate to their surroundings. ClimbingGraph solves this by transforming the Mountain Project dataset into a Knowledge Graph. By connecting routes directly to their locations and features, we use Graph Neural Networks and Embeddings to predict real difficulty levels and suggest personalized climbs based on structural similarity.

The following sections will explore how this project fulfills the specific learning outcomes through implementation and results.

Creation and Evolution of the Knowledge Graph

(LO7) Apply a system to create a Knowledge Graph

The creation of ClimbingGraph involved a multi-stage ingestion process using Python and Cypher.

I downloaded the Mountain Project dataset from Kaggle, which provides information on route names, difficulty, and physical dimensions. Using Python, I preprocessed the data to focus exclusively on California routes within the 5.7 to 5.12b difficulty range. This reduced the dataset to 7,302 routes, a size better suited for running Graph Neural Networks on a local machine.

For the geographic data, I converted the location strings into a clear hierarchy by establishing parent-child relationships between sectors, areas, and regions. I mapped routes to their specific sectors using the `IN_AREA` relationship and then built the spatial hierarchy using `SUB_AREA_OF` relationships. This structural creation was vital, as it transformed flat data into a connected topology where spatial proximity is represented by shared parent nodes in the graph. Once the data was structured, I saved it as CSV files ready to be imported into Neo4j.

Next, I loaded the cleaned data into Neo4j to build the Knowledge Graph. I defined two types of nodes, Routes and Locations, and linked them to form a clear hierarchy from specific sectors up to the state level. The final graph provides a structured network where routes and locations are connected through directed edges.

(LO8) Apply a system to evolve a Knowledge Graph

To handle missing data, I used a filling method in cypher. For the 577 routes without a recorded length, I assigned them the average length of other routes in that same sector. If

no other routes existed in that sector, I used 65 meters, which is the median for all California routes. This evolution step ensured that the GNN and embedding models received a complete feature matrix, significantly improving the stability of the training process.

The initial version of the graph consisted of 10,405 nodes and 10,583 edges. Because the number of nodes and edges was nearly identical, the graph followed a strict hierarchical tree structure. However, for a Graph Neural Network to perform effectively, it requires denser connectivity to pass information between nodes. To address this, I implemented a spatial proximity logic, connecting routes located within 100 meters of each other with a `NEAR_TO` relationship. This process added 45,905 new relationships, bringing the total to 56,488 edges. By moving beyond a simple tree structure and creating this dense network, I provided the GNN with the rich contextual information it needed to significantly improve its predictive metrics.

(LO1) Understand and apply Knowledge Graph Embeddings

To fulfill this learning outcome, I implemented Knowledge Graph Embeddings to translate the complex, networked relationships of the climbing world into a format a computer can understand. While a graph is great for representing connections, its irregular structure makes it difficult to compare routes directly using traditional algorithms. By using embeddings, I was able to compress each route's identity, including its technical difficulty, geographic location, and its specific neighborhood, into a dense vector.

By representing routes as points in a high-dimensional space, the system can calculate the mathematical distance between them. Routes that are structurally and technically similar end up closer together, allowing me to find the best recommendations based on their similarities. This transformation was essential not only for finding similar climbs but also for providing a clean, numerical input for the machine learning models used later in the project.

Implementation with hybrid Embeddings

For the technical implementation, I used the **FastRP** (Fast Random Projection) algorithm in Neo4j to generate 64-dimensional structural embeddings. I chose FastRP because it is highly efficient at capturing the local neighborhood structure of the routes. To ensure the embeddings didn't just reflect the routes' locations, I applied a high property weight (0.9) to technical features like rating, length, and coordinates. Even with this weighting, the graph's hierarchical connectivity meant that geographic location still heavily influenced the resulting vectors.

To solve this and give more importance to technical attributes like grade and length, I moved to Python to develop a hybrid embedding approach. I concatenated the 64D structural vector from Neo4j with a 4D normalized feature vector to create a more balanced representation. This vector included:

- **Normalized Rating:** Scaled against a maximum rating of 12.0. I applied a manual weight (multiplying by 2.0) to give difficulty a higher priority in similarity calculations.
- **Normalized Length:** Scaled by the maximum route height in the dataset.
- **Spatial Coordinates:** Min-Max normalized Latitude and Longitude to keep geographic data within a 0–1 range.

Results in Recommendation

By applying **Cosine Similarity** to these tensors, I built a recommender system that identifies the top 10 routes that are structurally and physically near a target route. Unlike a simple text search, this system can suggest a route that feels like the target route in terms of length and difficulty, while ensuring it remains within the same geographic area. This provides a solution to the problem of route discovery without requiring manual filtering of user comments.

(L03) Understand and apply Graph Neural Networks

Graph Neural Networks (GNNs) are specialized models designed to process data in a graph format, allowing them to learn from both individual nodes and the specific relationships between them. In the climbing world, difficulty grades are notoriously subjective, they are usually suggested by the person who first "equips" or climbs the route, which often leads to controversy, even among the world's elite climbers who struggle to agree on the difficulty of top-tier routes. By applying a GNN, we can move toward a more objective "true" grade by training the model to look at a route within the context of its neighbors.

This approach is especially effective because climbing sectors often follow physical trends that a graph can capture. For instance, it is common for routes in a single sector to become progressively harder from left to right as the wall gets steeper or the cliff gets higher. While human grades change from person to person, a GNN uses a route's position among its neighbors to predict a more objective and consistent difficulty level.

Implementation and Model Architecture

I implemented a **GraphSAGE** (Graph SAmple and aggreGAtE) architecture using PyTorch Geometric. GraphSAGE was chosen specifically for its inductive learning capabilities. In a real-world climbing app, new routes are added daily. Unlike other models, GraphSAGE can generate predictions for these unseen nodes by aggregating features from their local neighborhood without needing to retrain the entire graph.

Before training, I constructed a **hybrid feature matrix** to provide the GNN with a complete view of each route. Each node was represented by a 68-dimensional vector (similar to the previous LO), created by concatenating two distinct types of data:

- **64D "Clean" Structural Embeddings:** Generated via FastRP in Neo4j. To ensure the integrity of the model and prevent target leakage, these embeddings were computed strictly using graph topology and physical metadata, excluding the route's

rating. This forces the GNN to derive difficulty from connectivity and geographic patterns rather than simply reading a pre-existing grade attribute.

- **4D Physical Attributes:** Raw technical features including route length, community star rating, and normalized geographic coordinates (latitude and longitude).

To ensure numerical stability and faster convergence during training, I applied a **StandardScaler** to the entire feature matrix. This levels the playing field across all features. Without it, larger numbers like route length could dominate the more subtle embedding values, tricking the model into focusing only on the biggest numbers instead of learning from all 68 features equally.

I developed two distinct models to approach the problem:

1. **GNN Regressor:** Designed to predict the specific numerical grade. This model uses two SAGEConv layers with mean aggregation. The first layer compresses the 68 features into a 64-dimensional hidden space, followed by a second layer that reduces it to 32 before a linear output. I used **Mean Squared Error (MSE)** as the loss function
2. **Deep Level Classifier:** To provide broader, more practical categories for climbers, I built a deeper architecture with three SAGEConv layers. This model incorporates Batch Normalization after each layer to stabilize the gradients and allow for a higher learning rate. It classifies routes into three categories: Beginner (≤ 5.9), Intermediate (5.10a–5.11a), and Advanced ($\geq 5.11b$) using a Cross-Entropy loss function.

Performance and Evaluation

I initially trained the models using embeddings that included the rating property, which resulted in very high metrics. However, I soon realized this was essentially giving the model the answers. To ensure the results were authentic and useful for the real world, I regenerated the clean embeddings without any rating data. While this naturally led to a drop in performance, it provided a far more honest assessment of how well the model learns from graph structure alone.

With the honest dataset, the Regressor achieved a **Mean Absolute Error (MAE)** of **2.1313 grades**. Given the inherent subjectivity of climbing, being able to predict a grade within roughly two increments using only topology and basic physical features is a significant result. It proves the model can identify difficulty trends based on where a route is located.

For the classification task, I used the **Dice Score** (Macro-F1) to evaluate how well the model sorted routes into the three athletic categories. The model achieved a final **Dice Score of 0.6264**. Through message passing, the GNN recognizes that a route located in a notoriously steep or "Advanced" sector should likely have its difficulty adjusted upward, correcting for the original setter's bias based on the surrounding context.

Database Models and Architectures

(LO4) Compare different Knowledge Graph data models

I evaluated three distinct data models used across different technical communities. While the **Relational Model** using CSVs is efficient for storage, it lacks the connectivity

required for complex spatial reasoning. By adopting a **Property Graph Model** in Neo4j, I modeled hierarchies like "Route > Sector > Area" as native graph structures for efficient geographical processing. Finally, I utilized a **Latent Vector Model** by transforming the graph into 64D FastRP embeddings, enabling the GNN to process knowledge within a continuous mathematical space rather than through rigid symbolic rules.

(LO5) Design and implement architectures of a Knowledge Graph

The ClimbingGraph uses a hybrid architecture designed to scale across thousands of routes. I split the workload between two systems. Neo4j (with the GDS library) handles the data storage and graph projections, and Python with PyTorch runs the GNN and SAGEConv layers. This division of labor allows the database to manage the large-scale connectivity while the Python code focuses on the model training. By separating the tasks this way, I avoided memory bottlenecks and made the entire training process much more efficient.

Applications and Connections

(LO9) Describe and design real-world applications of Knowledge Graphs

ClimbingGraph is a practical application of Knowledge Graphs in the sports sector. It moves beyond basic keyword searches to create a recommender system that understands a climb's technical style (much like how industrial KGs identify similar components in manufacturing or logistics). This helps climbers discover new routes and better understand grading while they're actually out at the crag.

(LO10) Describe financial Knowledge Graph applications

While this project focuses on climbing, the logic of grading and neighbor analysis is directly applicable to finance. Just as ClimbingGraph identifies "sandbagged" routes by looking at sector neighbors, financial KGs can detect risky entities by analyzing their relationships to known fraudulent actors or unstable markets.

(LO11) Apply a system to provide services through a Knowledge Graph

ClimbingGraph offers two main services: predicting a route's difficulty and recommending what to climb next. By using the graph's structure and embeddings, the system can estimate a grade even when the information is missing or incorrect. It also powers a "Next Climb" service that uses the graph's logic to match a climber's skill with the specific features of a crag, turning raw data into a useful tool for the field.

(LO12) Describe the connections between KGs, ML, and AI

This project demonstrates that KGs are a melting pot of AI techniques. It bridges the gap between symbolic data (the geographic hierarchy and Cypher rules) and sub-symbolic ML (GNNs and embeddings). This shows how structured graph knowledge can guide neural networks to produce more accurate and context-aware results.

Reproducibility and Code Repository

To ensure the project's methodology and results are fully reproducible, I have documented the complete codebase and implementation steps in the following GitHub repository: <https://github.com/juanmooreno/climbing-knowledge-graph>.

The repository is organized to follow the project's workflow:

- **`preprocess.ipynb`**: Handles the initial data cleaning and transformation of the Kaggle dataset.
- **`neo4j_queries.cql`**: Includes all the queries used to build the graph hierarchy, handle missing data, and generate the FastRP projections within Neo4j.
- **`embeddings_and_gnns.ipynb`**: Contains the Python implementation for the hybrid embeddings and the GraphSAGE models.

By following the README instructions, you can recreate the ClimbingGraph environment and run the entire pipeline from raw data to final predictions.

Conclusion

The ClimbingGraph project shows that moving from flat CSV files to a connected Knowledge Graph in Neo4j completely changes how we can analyze climbing data. By building a spatial hierarchy, I created a system that understands the relationship between a route and its surroundings, rather than treating every entry as an isolated row. The most impactful part of the project was the hybrid embedding approach. By merging FastRP structural vectors with physical technical features, I generated a unique numerical fingerprint for every route. This allowed the recommender system to move beyond simple keyword searches and actually find routes that share a similar technical soul and geographic neighborhood.

My results, including a Mean Absolute Error of 2.13 grades and a Dice Score of 0.62, are a realistic reflection of the sport's inherent subjectivity. While these metrics are not perfect, they are honest. By avoiding target leakage, I ensured the system was learning from actual patterns rather than just reading the answer.

Looking ahead, there is a clear path to making these predictions even more accurate. The model currently works on a relatively small slice of data, but expanding the graph to include more regions would create the dense connectivity needed to sharpen the results. Adding more specific features to the embeddings, like rock type or wall steepness, would also help the system better understand the nuances of different climbing styles. This project isn't just about a single prediction, it's a proof of concept that shows how structured graph knowledge can turn subjective, messy data into a reliable tool for route discovery and assessment.